

Radiology AI

AI Disclosure Appendix

Fair Use of AI — Code of Conduct Compliance

V G Masilamani(DA25S005)

AI Tool Used: Claude (Anthropic) — claude.ai

Purpose: Conceptual clarification, syntax help, bug debugging,
UI boilerplate, and non-MLOps configuration assistance

Core MLOps Logic: Designed, implemented, and understood by the student

Compliance: Adheres to DA5402 Code of Conduct: Fair Use of AI

Contents

1	Overview of AI Usage	2
2	Detailed AI Usage Log	2
2.1	Category 1 — Conceptual Understanding (Allowed)	2
2.2	Category 2 — Syntax Clarification (Allowed)	3
2.3	Category 3 — Bug Debugging (Allowed)	3
2.4	Category 4 — Non-MLOps Boilerplate & UI (Allowed)	4
3	What Was NOT Done Using AI	4
4	Compliance Summary	5

Overview

of

AI

Usage

In accordance with the DA5402 Code of Conduct on Fair Use of AI, this appendix discloses all instances where Claude (Anthropic) was consulted during the development of the RadiologyAI project. AI assistance was limited to:

- **Conceptual understanding** — asking “what is data drift” or “how does Grad-CAM work”
- **Syntax clarification** — asking “what is the correct FastAPI syntax for file upload”
- **Bug debugging** — pasting error messages and asking for explanations
- **Non-MLOps boilerplate** — React UI components, nginx config, Docker networking
- **Documentation structuring** — proofreading and LaTeX formatting help

AI was **NOT** used for:

- Designing the MLOps pipeline architecture
- Writing the DVC pipeline stages or versioning strategy
- Designing the MLflow experiment tracking schema
- Designing the Airflow DAG logic or branching conditions
- Designing the Prometheus instrumentation or alert rules
- Writing the model training logic or hyperparameter choices
- Generating or fabricating any metrics or monitoring logs

Detailed

AI

Usage

Log

Category

1

—

Conceptual

Understanding

(Allowed)

These interactions were used to understand concepts before implementing them manually.

Prompt / Question Asked	How	I	Used
“Explain what data drift is and how Evidently AI detects it”	Used to understand the concept. Then manually chose which pixel stat		
“What is Grad-CAM and how does it work mathematically?”	Used to understand gradient-weighted class activation mapping. The		
“Explain the difference between DVC stages and DVC pipelines”	Used to understand how dvc.yaml works. Then design		
“What is the difference between MLflow autolog and manual logging?”	Used to understand what autolog captures automatically. Then decided		

Prompt / Question Asked	How I Used
“How does CosineAnnealingLR work in PyTorch?”	Used to understand the scheduler behaviour. Then chose the
“What is weighted cross-entropy loss and when should I use it?”	Used to understand class imbalance handling. Then manually comp
“What is the difference between Prometheus Counter, Gauge, and Histogram?”	Used to understand Prometheus metric types. Then independent
“Explain the Airflow XCom system”	Used to understand task communication. Then designed the branching
“What is TorchScript and why is it used for model serving?”	Used to understand the export format. Then
“What is the MLflow Model Registry and how does staging work?”	Used to understand Production/Staging/Archived stages.

Category 2 — Syntax Clarification (Allowed)

Prompt / Question Asked	How I Used
“What is the correct FastAPI syntax for accepting a file upload?”	Used to understand UploadFile and File() imports. Then wrote
“How do I use Pydantic BaseSettings to load from .env?”	Used to understand Pydantic Settings syntax. Then designed
“How do I use asynccontextmanager for FastAPI lifespan?”	Used to understand the lifespan pattern. Then wrote my own lifespan
“How do I mount a volume in docker-compose for a specific path?”	Used to understand volume syntax. Then wrote my own
“How do I use React useState and useEffect for API calls?”	Used to understand hook patterns. Then w
“How do I create a Prometheus Gauge and set its value?”	Used to understand prometheus_client syntax. Then independently

Category 3 — Bug Debugging (Allowed)

Error / Bug	What I Did
ModuleNotFoundError: No module named 'ml' when running train.py	AI explained the PYTHONPATH issue. I then set <code>export PYTH</code>
alembic.util.exc.CommandError: Can't locate revision MLflow DB error	AI explained the MLflow version mismatch. I then upgra
sqlite3.OperationalError: unable to open database file in Docker	AI explained Docker volume permissions. I t
Airflow container showing cd: /opt/radiologyai: No such file	AI explained volume mounting in Docker. I then fixed
npm error: package-lock.json out of sync in Docker build	AI explained the difference between npm ci and npm insta
DVC showing output does not exist after dvc remove accident	AI explained that dvc remove only removes tracking, n
Grafana disk panel showing no data on macOS	AI explained node_exporter filesystem differences on macOS
Feedback Prometheus gauge resetting to zero on Docker restart	AI explained in-memory vs persistent state. I then designed and imple

Category 4 — Non-MLOps Boilerplate & UI (Allowed)

These are non-MLOps components where using AI for boilerplate is acceptable per the Code of Conduct.

Area	What AI
React frontend UI components	AI provided basic React component structure (useState, useDropzone).
nginx.conf for Docker frontend	AI provided a basic nginx configuration. I then modif
ReportLab PDF generation syntax	AI explained ReportLab's SimpleDocTemplate API. I then designed t
Alertmanager YAML syntax	AI explained the route/receiver/inhibit structure. I then
LaTeX document formatting	AI helped with LaTeX table syntax and package usage.

What Was NOT Done Using AI

The following were entirely student-designed and implemented without AI generation:

Component	My Own work
DVC pipeline design	I decided the 9 stages, their dependencies, and the dvc.yaml structure based on the project requirements
MLflow tracking schema	I chose which metrics, params, and artifacts to log beyond autolog
Two-phase training strategy	I decided frozen vs unfrozen epochs, learning rates, and the rationale
Class weight computation	I computed Normal=2.197, Pneumonia=0.760, COVID19=0.814 from dataset counts
Airflow DAG branching logic	I designed the feedback accuracy check, drift check, and branching conditions
Prometheus metric design	I chose which metrics expose which information and at what granularity
Alert rule thresholds	I chose 5% error rate, 200ms latency, 80% accuracy as thresholds
Feedback loop architecture	I designed the full loop: feedback → gauge → alert → Airflow → retrain
Drift detection approach	I chose pixel statistics (mean, std, brightness) as drift features
All MLOps design decisions	Every design choice was made by me and can be explained during viva

Compliance

Summary

Code of Conduct Requirement	Compliant?
Human-in-the-Loop: Core logic and versioning strategy is my own	YES
AI used only for scaffolding and debugging	YES
AI Disclosure Appendix included	YES
Tool used is listed	YES — Claude (Anthropic)
Specific prompts are documented	YES — Section 2
Sections where AI output was used are listed	YES — Categories 1–4
Can explain any line of code or design choice	YES
No synthetic/fabricated metrics	YES — All metrics are real
No AI-written MLflow/DVC interaction scripts	YES
“The AI generated it” is NOT an acceptable viva answer	Understood and prepared